Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Introducción a la Programación y Computación 2

Inga. Claudia Liceth Rojas Morales
Ing. Marlon Antonio Pérez Türk
Ing. José Manuel Ruiz Juárez
Ing. Edwin Estuardo Zapeta Gómez
Ing. Dennis Stanley Barrios Gonzalez
Ing. Fernando José Paz González

Tutores de curso:
Isai Eliezer Magdiel Molina Guevara
Eddy Alejandro Murga Barillas
Daniel Eduardo Velásquez Avila
Javier Alejandro Matías Guarcas
Jorge Andrés Mejía Suchite
Carlos Manuel Lima y Lima

PROYECTO No. 2

OBJETIVO GENERAL

Modelar, documentar e implementar una solución al problema que se plantea utilizando las herramientas de desarrollo presentadas en clase y laboratorio.

OBJETIVOS ESPECÍFICOS

- Implementar una solución utilizando el lenguaje de programación C#.
- Utilizar estructuras de programación secuenciales, cíclicas y condicionales.
- Generar reportes con la herramienta Graphviz.
- Manipular archivos XML.
- Utilizar los conceptos de TDA no lineales y aplicarlos a memoria dinámica.
- Utilizar estructuras de programación propias.
- Utilizar el paradigma de programación orientada a objetos.

DESCRIPCIÓN DEL PROBLEMA

Una cadena de librerías desea desarrollar un sistema inteligente para administrar más de 100,000 libros distribuidos en varias sucursales. El sistema debe permitir registrar nuevos libros, buscar títulos rápidamente, conocer qué libros están relacionados con una categoría específica y mostrar la organización completa del catálogo.

Actualmente, los empleados pierden demasiado tiempo porque los libros se encuentran registrados en listas extensas. Cuando un cliente solicita un libro, el sistema tarda en localizarlo y en determinar qué otros libros pertenecen a la misma temática.

La gerencia solicita un sistema que funcione de manera similar a una **biblioteca física organizada por secciones y estanterías**. Imaginemos que al entrar a la biblioteca se observa un gran mapa general; de este mapa se derivan áreas principales, luego categorías más específicas y finalmente los libros individuales. Además, cuando un cliente solicita un libro por su código o título, el sistema debe ser capaz de encontrarlo rápidamente sin revisar toda la colección.

Cada categoría o subcategoría posee un nombre, el listado de subcategorías internas y un conjunto de libros asociados a la misma.

Todo libro posee un número ISBN que representa un valor entero único para identificar cada libro, además posee un título, un autor y la categoría en que está clasificado.

Funcionalidades requeridas

1. Registrar nuevos libros en el catálogo.
2. Buscar un libro utilizando su ISBN (código único).
3. Mostrar la organización completa de las categorías y subcategorías de la librería.
4. Identificar qué libros pertenecen a una categoría determinada.
5. Obtener el libro con el código más pequeño y el más grande del catálogo.
6. Mostrar los libros en orden ascendente de ISBN.
7. Eliminar libros que ya no se encuentren disponibles.
8. Mantener tiempos de búsqueda eficientes incluso cuando el catálogo crezca significativamente.
9. Permitir visualizar la estructura organizativa del catálogo desde las categorías generales hasta los elementos más específicos, con opción a visualizar a partir de una subcategoría específica.

Restricciones

- El sistema debe mantener tiempos de búsqueda reducidos aun cuando existan miles de registros.
- Las categorías pueden contener subcategorías y estas, a su vez, otras divisiones internas.
- Las categorías deberán presentarse ordenadas alfabéticamente.
- Cada libro posee un ISBN que es un código único que lo identifica.

- La estructura debe representar relaciones jerárquicas similares a las de una biblioteca, un organigrama o un sistema de carpetas.

Usted ha sido contratado para diseñar una solución que permita representar la organización jerárquica del catálogo y realizar búsquedas rápidas de libros utilizando mecanismos adecuados para gestionar relaciones de dependencia, clasificación y localización eficiente de información.

ENTRADA

Se podrá utilizar un archivo en formato XML para crear una configuración inicial que facilite la evaluación de la funcionalidad del software desarrollado.

entrada.xml

```
<?xml version="1.0"?>
<config>
  <listaCategorias>
     <categoria padre=”nombreCategoriaPadre”[1]> [valorAlfanumerico] </categoria>
     ...
  </listaCategorias>
  <listaLibros>
    <libro>
     <ISBN>[valorNumerico]</ISBN>
     <titulo>[valorAlvanumerico]</titulo>
     <autor>[valorAlvanumerico]</autor>
     <categoria>[valorAlvanumerico]</categoria>
    </libro>
    ...
  </listaLibros>
</config>
```

Tomar en cuenta que este archivo es incremental, es decir, se pueden ingresar varios archivos de entrada, de tal manera que se pueden crear nuevas categorías, y nuevos libros, por lo que los elementos listaCategorias y listaLibros son opcionales.

NOTA: Los nombres de las categorías son únicos, es decir, no pueden haber categorías a un mismo nivel o a distintos niveles con el mismo nombre.

[1] Este atributo es opcional, ya que la categoría principal no posee padre.

INTERFAZ DE USUARIO

Se debe crear una interfaz de usuario fácil de utilizar e intuitiva que permita al usuario realizar las siguientes funciones:

a. Inicialización – Para que el sistema pueda inicializarse sin ninguna información previa.
b. Cargar un archivo XML de entrada
c. Gestión de catálogo
    a. Mostrar estructura de categorías y subcategorías
    b. Mostrar gráficamente los libros de una categoría o subcategoría (orden ascendente por ISBN) (utilizando Graphiz)
    c. Agregar una categoría
d. Gestión de libros
    a. Registrar un nuevo libro
    b. Eliminar un libro que ya no está disponible
    c. Mostrar libro con el menor ISBN
    d. Mostrar libro con el mayor ISBN
    e. Mostrar datos de un libro en base a su ISBN
e. Ayuda – Mostrar la información del estudiante y un link hacia la documentación del proyecto.
f. Debe ser implementada utilizando una interfaz web.

CONSIDERACIONES

Se deberá realizar la implementación utilizando programación orientada a objetos, algoritmos desarrollados por el estudiante e implementación de estructuras a través de Tipos de Dato Abstracto (TDAs) propios del estudiante, es decir, **no deben utilizar estructuras de datos propias de C# (ListedLink, List, Queue, Stack, templates etc.)**. El estudiante deberá abstraer la información y definir qué estructuras implementar para construir la solución.

Debe utilizarse versionamiento para el desarrollo del proyecto. Se utilizará la plataforma **Github** en la cual se debe crear un repositorio en el que se gestionará el proyecto. Se deben realizar **4 releases** o versiones del proyecto (Se sugiere desarrollar un Release por semana). **Se deberá agregar a sus respectivos auxiliares como colaboradores del repositorio**. El último release será el release final y se deberá de realizar antes de entregar el proyecto en la fecha estipulada.

DOCUMENTACIÓN

Para que el proyecto sea calificado, el estudiante deberá entregar la documentación utilizando el formato de ensayo definido para el curso. En el caso del proyecto, el ensayo puede tener un mínimo de 4 y un máximo de 7 páginas de contenido, este máximo no incluye los apéndices o anexos donde se pueden mostrar modelos y diseños utilizados para construir la solución. Es obligatorio incluir el diagrama de clases del diseño del software desarrollado y se recomienda incluir el modelo conceptual y los diagramas de actividades que modelan la solución de software presentada por el estudiante.

RESTRICCIONES

- Solo se permitirá la utilización de los IDEs discutidos en el laboratorio.
- Uso de TDA implementados por el estudiante obligatorio; el uso de estructuras de C# (ListedLink, List, Queue, Stack, Templates, etc.) resultará en penalización del 100% de la nota.
- Uso obligatorio de programación orientada a objetos (POO) desarrollada por completo por el estudiante. De no cumplir con la restricción, no se tendrá derecho a calificación
- El nombre del repositorio debe de ser **IPC2_Proy02_202602_#Carnet.**
- El estudiante debe entregar la documentación solicitada para poder optar a la calificación.
- Los archivos de entrada no podrán modificarse.
- Deben existir 4 releases mínimo, podría ser uno por cada semana, de esta manera se corrobora el avance continuo del proyecto.
- Se calificará de los cambios realizados en el último release hasta la fecha de entrega. Los cambios realizados después de ese release no se tomarán en cuenta.
- Cualquier caso de copia parcial o total tendrá una nota de 0 y será reportada a Escuela.
- Para dudas concernientes al proyecto se utilizarán los foros en UEDI de manera que todos los estudiantes puedan ver las preguntas y las posteriores respuestas.
- **NO HABRÁ PRÓRROGA.**

Usuarios GitHub de los auxiliares

- **Sección A - Eddy2109**
- **Sección B - EliezerGG**
- **Sección C - DanielVelAv**
- **Sección D - javiermatg**
- **Sección N - jorgemejia25**
- **Sección P - manuelimal25-dotcom**

ENTREGA

- La entrega será el jueves **24 de septiembre,** a más tardar a las 11:59 pm.
- La entrega será por medio de la UEDI.
- La documentación debe estar subida en el repositorio en una carpeta separada.
- Para entregar el proyecto en UEDI se deberá subir un archivo de texto con el link del repositorio.